02

Ensembles



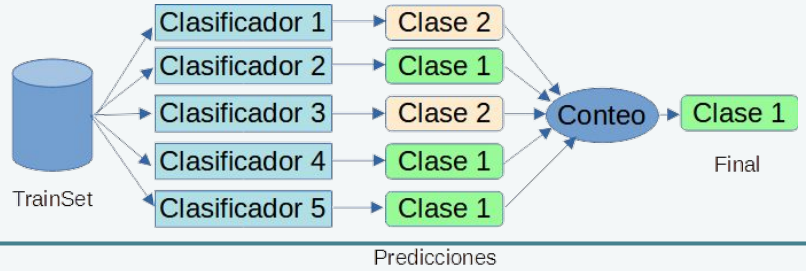
Ensembles

Los ensambles (ensembles) consiste en un conjunto de modelos clasificadores individuales (conocidos como estimadores base), los cuales todos juntos aportan al mecanismo de predicción.

Los ensambles conforman una solución que supera los desafíos técnicos que implica la creación de un único estimador.

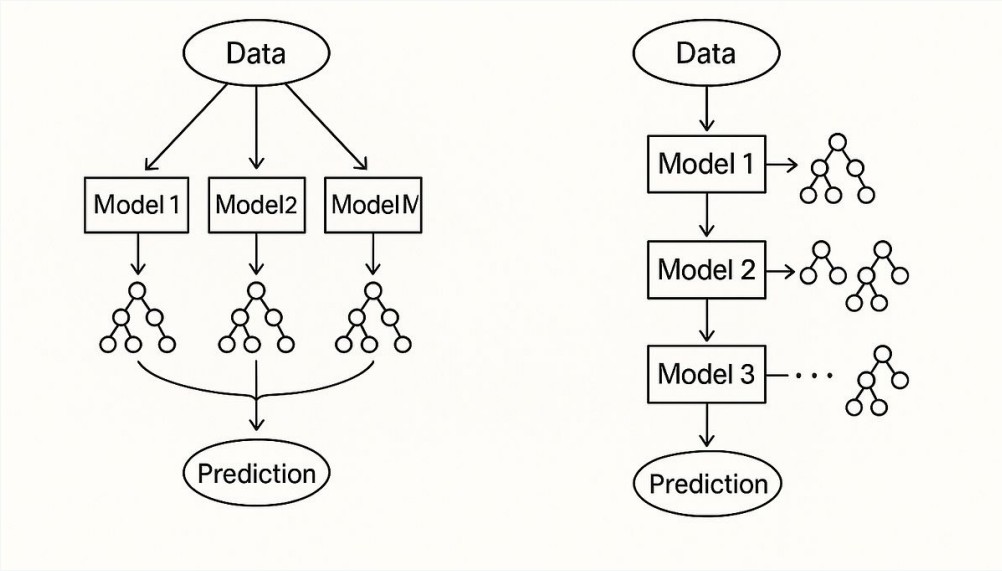
Ventajas:

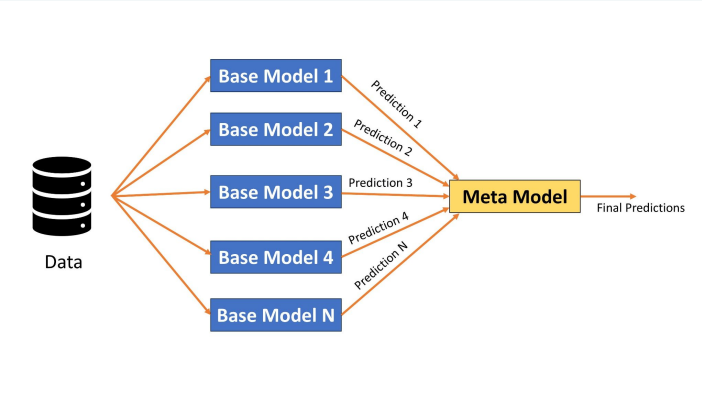
- Reducen el error en la predicción.
- Mantienen una buena generalización.



Ensembles

- Bagging
  - Random forest
- Boosting
  - AdaBoost
  - Gradient boosting
  - XGBoost
- Stacking

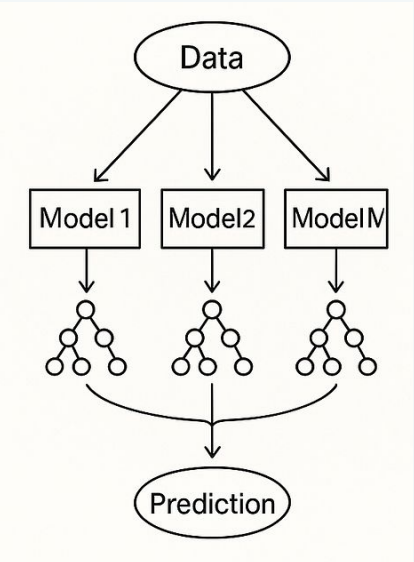




03

Bagging

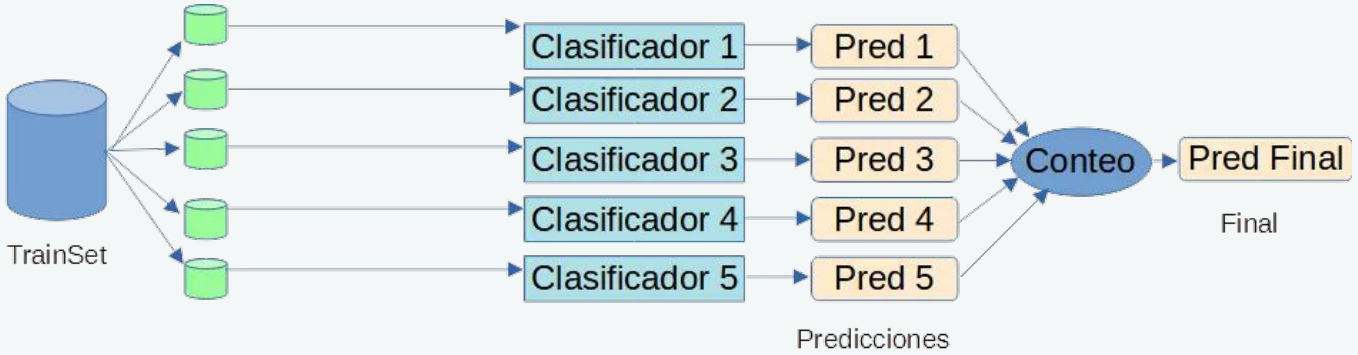
Random Forest



Bagging

- Se entrenan N estimadores diferentes.
  - Bootstrapping: se seleccionan al azar las filas del dataset (con reemplazo).
  - Pasting: se seleccionan al azar las filas del dataset (sin reemplazo).
  - Random subspace: se seleccionan al azar las columnas (atributos) del dataset.

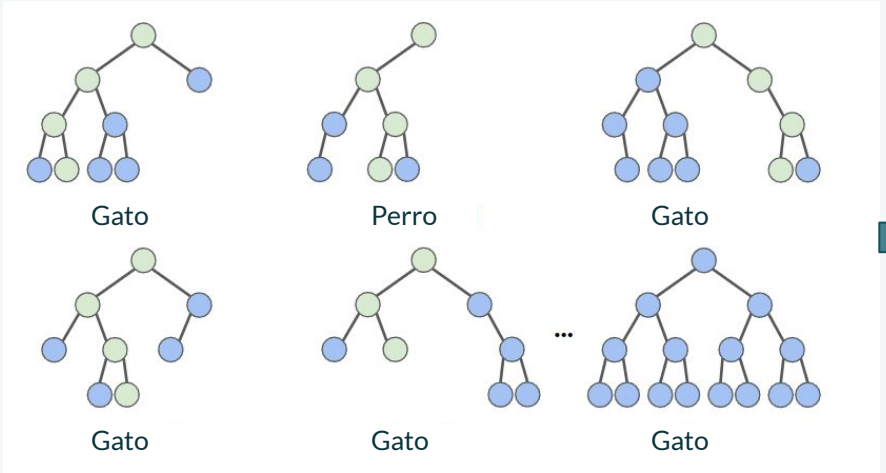
- La respuesta más votada es la respuesta del ensamble.



Random Forest

- Es un ensamble de **N** árboles independientes.
- Cada árbol se construye con un número distinto de muestras y de atributos.
- La predicción se realiza con el mecanismo de votación.

Peso: 6000 gr.
Salto: 180 cm.
Dientes: 28
Comida: 350 gr.



Gato (%)

Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators = 1000)
model = rf.fit(X_train, T_train)

T_pred = model.predict(X_test)

e = rf.estimators_[0]  # permite obtener los árboles (zero-based),
                       # por ejemplo para visualizarlos.

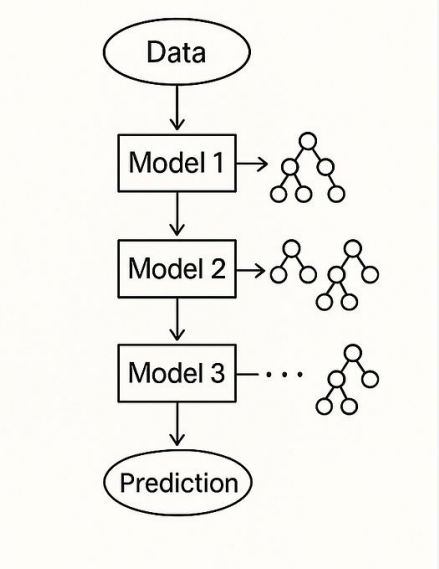
# devuelve la proporción de votos
print(model.predict_proba(X_test))

# https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
```

```
[0.13 0.87]
[1.   0.  ]
[0.88 0.12]
[0.51 0.49]
[0.1  0.9 ]
[1.   0.  ]
[0.96 0.04]
[0.89 0.11]
[0.16 0.84]
[0.18 0.82]
[0.1  0.9 ]
[0.14 0.86]
[0.05 0.95]
[0.99 0.01]
[0.42 0.58]
[0.46 0.54]
[0.04 0.96]
[0.91 0.09]
[0.98 0.02]
[0.77 0.23]
[0.61 0.39]
```

04

Boosting

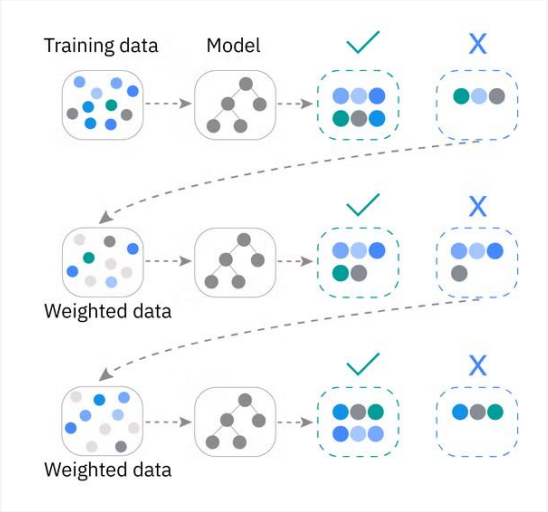


Boosting

El objetivo es usar varios modelos “débiles” para lograr un modelo “fuerte” que tenga un error mínimo.

La forma de armar los modelos es secuencial, uno tras otro, de tal manera que al entrenar un modelo, éste toma en cuenta los errores de su predecesor.

- AdaBoost
- Gradient Boosting
- XGBoost



AdaBoost (*Adaptative boosting*)

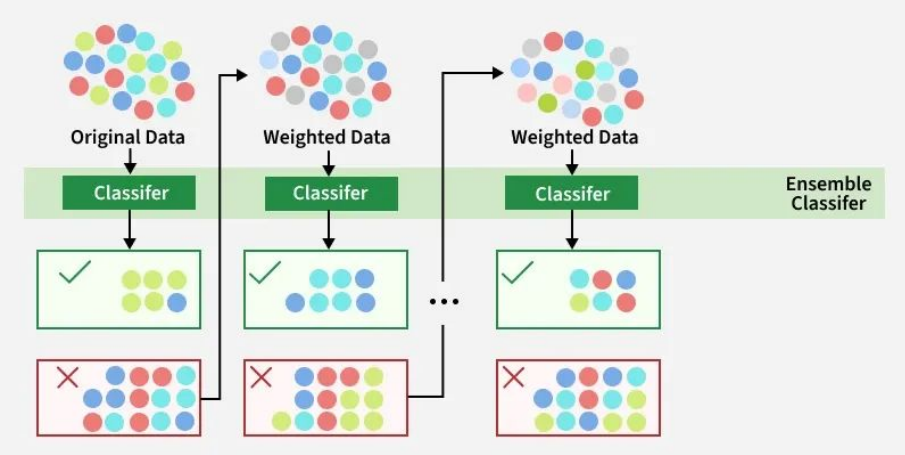
Al comienzo todas las muestras tienen el mismo “peso”.

Se entrena un primer modelo, el cual clasificará correctamente algunas muestras, mientras que otras no.

Las muestras mal clasificadas aumentan su “peso”.

El siguiente modelo a entrenar recibe las mismas muestras, con los pesos modificados. Nuevamente, las mal clasificadas aumentan su “peso”.

Se continúa de la misma manera un número prefijado de pasos...



AdaBoost (*Adaptative boosting*)

Durante el entrenamiento cada modelo obtuvo un “puntaje” de importancia α, según el nivel de precisión alcanzado.

Un modelo con buena performance, obtiene un valor alto de α.

Al momento de realizar la predicción, cada estimador vota de manera independiente, pero la votación se realiza ponderando la respuesta del modelo con su “puntaje” de importancia.

$$voto = \sum \alpha . pred_i(X)$$

AdaBoost

```
from sklearn.ensemble import AdaBoostClassifier

base = DecisionTreeClassifier()

ada = AdaBoostClassifier(estimator=base, n_estimators=500)
model = ada.fit(X_train, T_train)

print(model.estimator_weights_)      # permite obtener el peso de cada
                                     # estimador

print(model.predict_proba(X_test))   # devuelve la proporción de votos

# https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html
```

Gradient Boosting

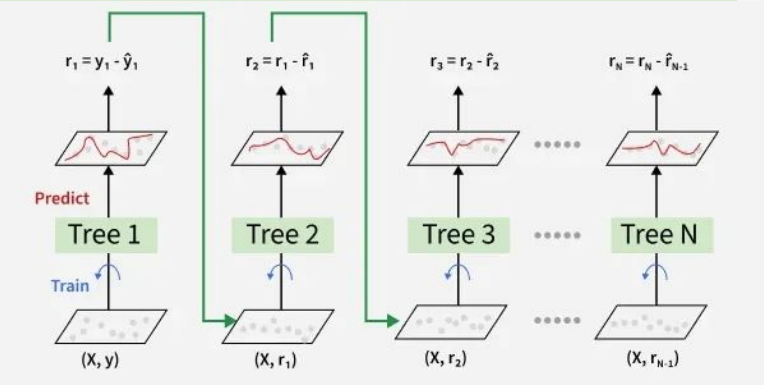
Se entrena un primer modelo (por lo general árboles de decisión), el cual cometerá errores en la clasificación.

Para cada muestra se calcula el “residuo”, que es el error (numérico) que cometió el estimador al hacer la predicción.

Los siguientes modelos “débiles” intentarán predecir esos “residuos”.

Se continúa de la misma manera un número prefijado de pasos.

El resultado es un modelo ensemble “fuerte”.



Gradient Boosting

Se comienza armando un primer clasificador débil, el más simple de todos.

Calcula la probabilidad de que una muestra sea de la clase “1”.

$$Prob_{inicial} = \frac{número_de_muestras_clase_1}{total_de_muestras}$$

También se calcula lo que se conoce como log(odds).

$$\log(odds)_{base} = \ln\left(\frac{número_de_muestras_clase_1}{número_de_muestras_clase_0}\right)$$

No	X	Y	T	Prob. inicial	log (odds) base
1	40	52	1	0,625	0,510
2	51	69	1	0,625	0,510
3	28	14	1	0,625	0,510
4	61	72	1	0,625	0,510
5	59	34	1	0,625	0,510
6	14	85	0	0,625	0,510
7	30	73	0	0,625	0,510
8	17	69	0	0,625	0,510

$$Prob_{inicial} = \frac{5}{9} = 0625 \qquad \log(odds)_{base} = \ln\left(\frac{5}{3}\right) = 0.510$$

Gradient Boosting

Se calculan los residuos (error cometido por el modelo).

$$Residuo = Target - Predicción$$

El residuo será utilizado para crear los siguientes modelos “débiles”

No	X	Y	T	Prob. inicial	residuo
1	40	52	1	0,625	0,375
2	51	69	1	0,625	0,375
3	28	14	1	0,625	0,375
4	61	72	1	0,625	0,375
5	59	34	1	0,625	0,375
6	14	85	0	0,625	-0,625
7	30	73	0	0,625	-0,625
8	17	69	0	0,625	-0,625

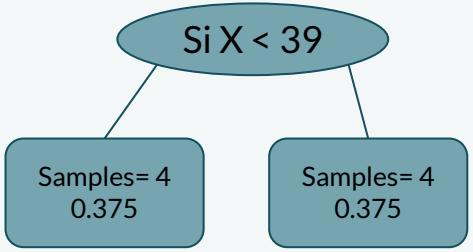
Gradient Boosting

Se genera el siguiente modelo “débil”. Este modelo es un árbol de regresión, cuyo objetivo es predecir el residuo de cada muestra.

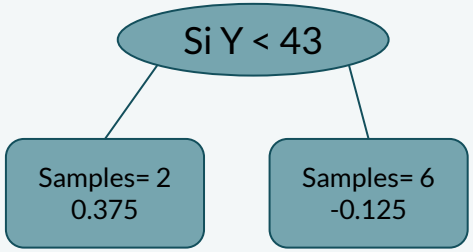
Para cada atributo del dataset, se calcula la función de pérdida (el error total que comete el modelo en todas sus predicciones). En cada nodo de decisión se elige el atributo y su valor que minimice la función de pérdida.

El Error Cuadrático Medio es uno de los más utilizados, pero hay otros.

$$ECM = \sum_{i=1}^{n} (Target - Predicción)^2$$



MCE=0.75



MCE=1.5

Gradient Boosting

Una vez seleccionado la condición del nodo de decisión se calcula el “valor” de cada hoja.

$$Valor = \frac{\sum_{i=1}^{M} Residuo_i}{\sum_{i=1}^{M} (Prob_{previ\,a_i} * (1 - Prob_{previ\,a_i}))}$$

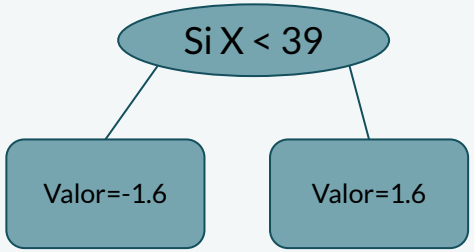
Luego se calculan las nuevas log(odds) y probabilidades de cada muestra.

$$\log(odds)_i = \log(odds)_i + Valor_i * learning_rate$$

$$Prob_i = \frac{1}{1 + \exp^{-\log(odds)_i}}$$

learning_rate es un hiperparámetro del algoritmo.

No	X	Y	T	log (odds)	Prob.
1	40	52	1	1,310	0,787
2	51	69	1	1,310	0,787
3	28	14	1	-0,289	0,428
4	61	72	1	1,310	0,787
5	59	34	1	1,310	0,787
6	14	85	0	-0,289	0,428
7	30	73	0	-0,289	0,428
8	17	69	0	-0,289	0,428



Gradient Boosting

Con las probabilidades se calculan los nuevos residuos de cada muestra.

Los nuevos residuos se utilizan para entrenar el siguiente modelo débil.

Por lo general, los árboles de decisión se los crea con un máximo de hojas entre 8 y 32, pero se puede utilizar cualquier otro criterio de parada.

No	X	Y	T	Prob.	Residuo
1	40	52	1	0,787	0,212
2	51	69	1	0,787	0,212
3	28	14	1	0,428	0,571
4	61	72	1	0,787	0,212
5	59	34	1	0,787	0,212
6	14	85	0	0,428	-0,428
7	30	73	0	0,428	-0,428
8	17	69	0	0,428	-0,428

Gradient Boosting

Con cada nuevo modelo se vuelven a calcular las nuevas probabilidades, y el algoritmo continúa hasta entrenar la cantidad de modelos prefijada.

No	X	Y	T	$Prob_{inicial}$	$Prob_1$	$Prob_2$	$Prob_3$	$Prob_4$	$Prob_5$	$Prob_6$
1	40	52	1	0,625	0,787	0,874	0,929	0,961	0,979	0,988
2	51	69	1	0,625	0,787	0,874	0,929	0,961	0,979	0,988
3	28	14	1	0,625	0,428	0,342	0,265	0,200	0,148	0,108
4	61	72	1	0,625	0,787	0,874	0,929	0,961	0,979	0,988
5	59	34	1	0,625	0,787	0,874	0,929	0,961	0,979	0,988
6	14	85	0	0,625	0,428	0,342	0,265	0,200	0,148	0,108
7	30	73	0	0,625	0,428	0,342	0,265	0,200	0,148	0,108
8	17	69	0	0,625	0,428	0,342	0,265	0,200	0,148	0,108

Gradient Boosting

Para realizar la predicción se utilizan los log(odds) y probabilidades que aporta cada modelo “débil”.

$$pred = \log(odds)_{base} + \sum_{i=1}^{T} Valor_{arbol_i} * learning_rate$$

$$Prob = \frac{1}{1 + e^{-pred}}$$

No	X	Y	pred	Prob	Respuesta
101	61	37	4,085	0,983	1
102	17	69	-0,344	0,414	0

Se redondean las probabilidades

Gradient Boosting

```
from sklearn.ensemble import GradientBoostingClassifier

gbc = GradientBoostingClassifier(loss='log_loss', learning_rate=0.1,
                                 max_depth=3, n_estimators=50)
model = gbc.fit(X_train, T_train)

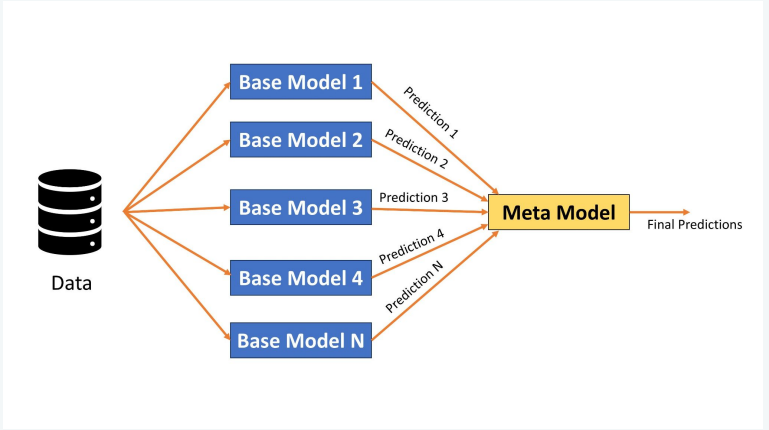
print(model.predict_proba(X_test))   # devuelve la proporción de votos

# https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html
```

```
# Otras opciones
import lightgbm as lgb
lgbm = lgb.LGBMClassifier()
from xgboost import XGBClassifier
xgb = XGBClassifier()
```

05

Stacking


Base Model 1
Base Model 2
Base Model 3
Base Model 4
Base Model N
Prediction 1
Prediction 2
Prediction 3
Prediction 4
Prediction N
Meta Model
Final Predictions
Data

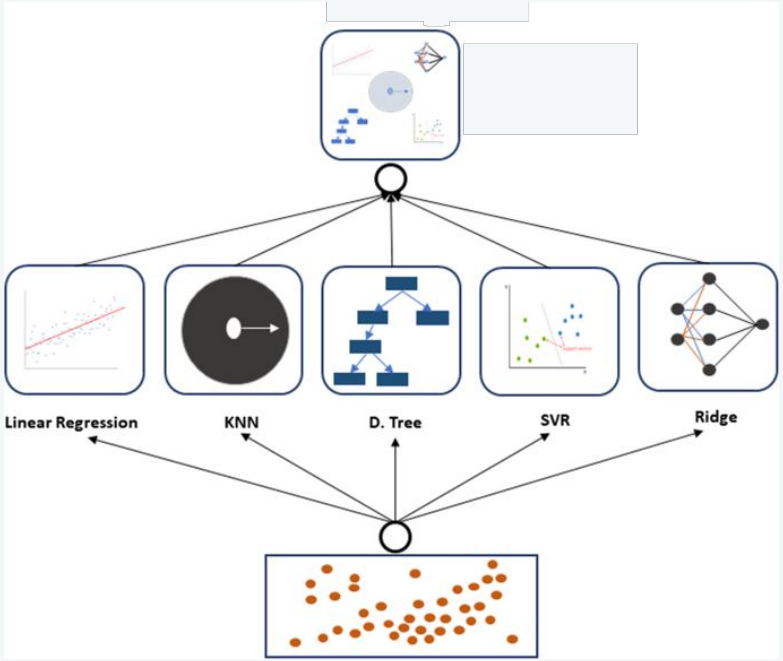
Stacking

El método de ensambles Stacking plantea el uso de múltiples estimadores de naturaleza distinta: árboles de decisión, modelos de regresión, SVM, Naïve Bayes, redes neuronales, etc.

Estos estimadores son llamados “base” o “débiles”.

Introduce el concepto de meta-aprendizaje, esto es un modelo que aprende lo aprendido por el resto de los modelos.

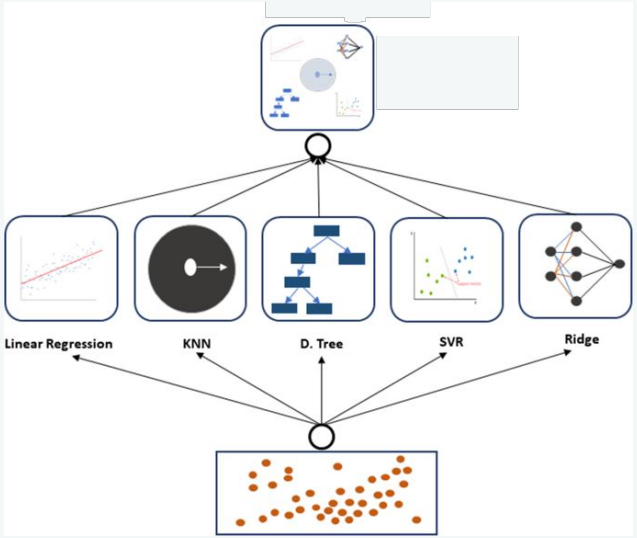
El “meta-estimador” aprende de las predicciones de los estimadores de “base”.



Stacking

Todos los estimadores aprenden de manera independiente.

Se arma un nuevo dataset, el cual está conformado con las predicciones de cada estimador.

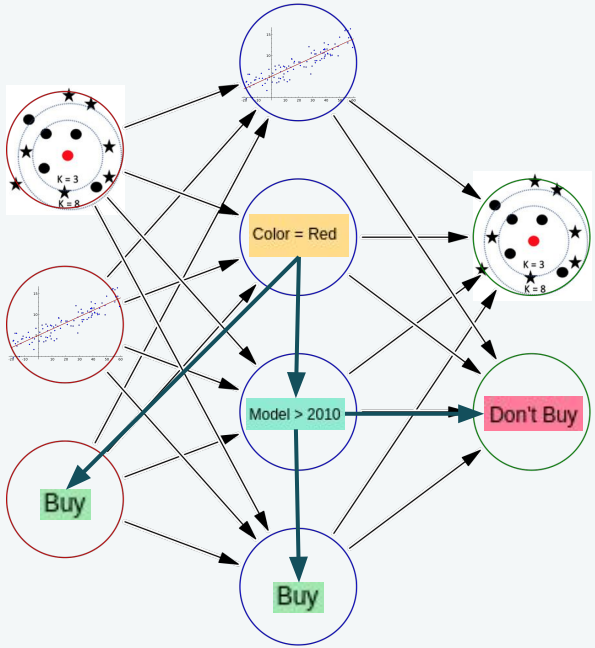


ID	X	Y	Z	T	Pred. RL	Pred. SVM	Pred. RF	Pred. RN	Pred. KNN
1	3.345	43.71	-1.73	1	0.934	0.87	0.23	0.19	0.892
2	7.21	57.2	-3.81	0	0.231	0.143	0.04	0.931	0.81
3	1.98	32.89	1.83	1	0.45	0.76	0.59	0.289	0.512
4	-0.29	85.23	2.21	0	0.001	0.128	0.014	0.003	0.109

Stacking

Como último paso se entrena el “meta-estimador” que tome como entrada las predicciones de los estimadores de “base”, para predecir la clase de cada muestra. El meta-estimador puede ser cualquiera de todos los disponibles (¡¡¡ incluyendo un ensemble !!!!)

ID	T	Pred. RL	Pred. SVM	Pred. RF	Pred. RN	Pred. KNN	Pred
1	1	0.934	0.87	0.23	0.19	0.892	0.81
2	0	0.231	0.143	0.04	0.931	0.81	0.35
3	1	0.45	0.76	0.59	0.289	0.512	0.63
4	0	0.001	0.128	0.014	0.003	0.109	0.072



Stacking

```
from sklearn.ensemble import StackingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression

# Crear los estimadores base, cada uno configurado como sea necesario
base_learners = [('bl1', DecisionTreeClassifier(max_depth=4)),
                 ('bl2', KNeighborsClassifier(n_neighbors=5)),
                 ('bl3', LogisticRegression()) ]

stck = StackingClassifier(estimators = base_learners,
                          final_estimator = RandomForestClassifier())
model = stck.fit(X_train, T_train)

print(model.predict_proba(X_test))   # devuelve la proporción de votos

# https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html
```

Stacking de múltiples capas

```
from sklearn.ensemble import StackingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.linear_model import LinearRegression

base_learners_1 = [('bl1', DecisionTreeClassifier(max_depth=4)),
                   ('bl2', LogisticRegression()) ]
base_learners_2 = [('bl3', KNeighborsClassifier(n_neighbors=5)),
                   ('bl4', LinearRegression()) ]

stck_2 = StackingClassifier(estimators = base_learners_2,
                            final_estimator = RandomForestClassifier())
stck_1 = StackingClassifier(estimators = base_learners_1,
                            final_estimator = stck_2)
model = stck_1.fit(X_train, T_train)
```

¿Cómo está conformado este stacking?

Stacking

```
from sklearn.ensemble import StackingClassifier

stck = StackingClassifier(estimators = base_learners,
                          final_estimator = RandomForestClassifier(),
                          cv = 10,
                          passthrough = True)
model = stck.fit(X_train, T_train)
```

cv → Cross validation
passthrough → en True, además de las predicciones , usa las muestras originales.

Actividades

2. Utilice la base de datos “glass”, separando en 90-10.

   Entrene un AdaBoost, un Gradient Boosting y un Stacking (con al menos cuatro estimadores “base”).

   Calcule e imprima accuracy, precision, recall, f-score y AUC-ROC.

   Repita los cálculos 10 veces, cambiando el random_state en el split. Calcule el promedio e IC_95% de cada métrica.

   Compare los promedios e IC obtenidos con los de la actividad 1.

RESUMEN

- Métricas de evaluación
- Ensembles
- Bagging
- Boosting
- Stacking

